VARDHAMAN COLLEGE OF ENGINEERING

(Autonomous)

Hyderabad, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A COURSE END PROJECT REPORT ON

MULTITHREADING AND SYNCHRONISATION

OPERATING SYSTEMS LABORATORY - A9510

Submitted by:

G. YASHASWINI REDDY	25881A0561
G. YASHWANTH	25881A0562
D. YOGESH	25881A0563

Batch No: 20

Course Facilitator

Ms. Kavitha Chitrala
Assistant Professor, CSE



VARDHAMAN COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2026–27

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH**,
Approved by **AICTE**, Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad - 501 218,
Telangana, India.

Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Course End Project titled **"Multithreading and Synchronisation"** is carried out by **"Gaddam Yashaswini Reddy", "Gangarapu Yashwanth"** and **"Deekonda Yogesh"** with Roll Numbers **"25881A0561", "25881A0562"** and **"25881A0563"** respectively (Batch No. 20) towards **A9510 – Operating Systems Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic Year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Kavitha Chitrala	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalatha**
Assistant Professor, CSE	Professor, CSE	Department of CSE

Contents

1 Introduction **5**
1.1 Problem Statement . . . 5
1.2 Objectives . . . 5
1.3 Scope . . . 5

2 Literature Review **6**

3 Description of Concepts **6**
3.1 Multithreading . . . 6
3.1.1 Advantages of Multithreading . . . 6
3.2 Race Condition and Critical Section . . . 7
3.3 Mutex . . . 7
3.4 Semaphore . . . 7
3.5 Condition Variable . . . 8

4 Methodology and System Design **8**
4.1 Approach . . . 8
4.2 Algorithm (Producer–Consumer) . . . 9
4.3 Multithreading . . . 9
4.4 Race Condition and Critical Section . . . 9
4.5 Mutex . . . 10
4.6 Semaphore . . . 10
4.7 Condition Variable . . . 10

5 Implementation **10**
5.1 Module 1: Race Condition and Mutex . . . 10
5.2 Module 2: Producer–Consumer using Semaphores . . . 11
5.3 Module 3: Condition Variables . . . 12
5.4 Module 4: Dining Philosophers (Deadlock Avoidance) . . . 13
5.5 Module 5: Readers–Writers Problem . . . 15

6 Results and Output **16**
6.1 Module 1 Output . . . 16
6.2 Module 2 Output . . . 17
6.3 Module 3 Output . . . 17
6.4 Module 4 Output . . . 18
6.5 Module 5 Output . . . 19
6.6 Comparison Table . . . 19

7 Mapping POs and SDGs **21**
7.1 Program Outcomes (POs) Mapping . . . 21
7.2 Sustainable Development Goals (SDGs) Mapping . . . 21

8 Conclusion **22**

9 Future Work **22**

10 Bibliography **23**

List of Figures

1 Race Condition 16
2 producer and consumer 17
3 condition variable demo 17
4 Dining Philosophers 18
5 Readers-Writers 19

List of Tables

1 Thread vs Process 7
2 Hardware and Software Requirements 8
3 Thread vs Process 9
4 Observed results of Module 1 (8 threads × 1,000,000 increments) 19
5 Summary of project programs and their results 20
6 Mapping of Program Outcomes to Project Relevance 21
7 Mapping of SDGs to Project Relevance 21

Abstract

Multithreading is an important concept in operating systems that allows multiple threads to execute concurrently within a single process, improving system performance and resource utilization. However, when multiple threads access shared resources simultaneously, synchronization mechanisms are required to prevent data inconsistency and race conditions. This project demonstrates the implementation of multithreading and synchronization concepts using the C programming language. Multiple threads are created to perform concurrent tasks while accessing shared resources. Synchronization techniques such as mutexes, semaphores, and condition variables are employed to ensure safe and coordinated access to these resources. Mutexes provide mutual exclusion, semaphores control resource availability, and condition variables facilitate communication and coordination between threads.

Keywords: Threads, Mutex, Semaphore, Condition Variable, Race Condition, Critical Section, Deadlock, POSIX.

1 Introduction

Multithreading is a fundamental concept in operating systems that enables multiple threads to execute concurrently within a single process. A thread is the smallest unit of CPU execution and shares the process's code, data, and memory resources while maintaining its own program counter, registers, and stack. Multithreading improves application responsiveness, resource utilization, and overall system performance, especially on multicore processors.

When multiple threads access shared resources simultaneously, synchronization mechanisms are required to ensure correct execution and prevent inconsistencies. Without proper synchronization, concurrent access can lead to race conditions, deadlocks, and data corruption. Operating systems provide synchronization tools such as mutexes, semaphores, and condition variables to coordinate thread execution and protect shared resources. This project focuses on demonstrating multithreading and synchronization concepts using these mechanisms to ensure safe and efficient concurrent execution.

1.1 Problem Statement

To develop a system that demonstrates multithreading and synchronization concepts in an operating system environment by creating multiple threads and managing access to shared resources using mutexes, semaphores, and condition variables. The project aims to illustrate how synchronization mechanisms prevent race conditions and ensure correct thread coordination.

1.2 Objectives

- To understand the concept of multithreading in operating systems.
- To create and manage multiple threads within a process.
- To study synchronization issues arising from concurrent execution.
- To demonstrate the use of mutexes for mutual exclusion.
- To implement semaphores for resource management and thread synchronization.

1.3 Scope

The project focuses on user-level multithreading and synchronization concepts implemented using the C programming language and POSIX thread libraries. It covers the practical use of mutexes, semaphores, and condition variables for managing access to shared resources. The project is designed for educational purposes and provides insights into concurrency management techniques used in operating systems, distributed systems, and parallel computing applications.

2 Literature Review

The concepts of multithreading and synchronization have been extensively studied in operating system research and practice. One of the primary references for this project is *Operating System Concepts* by Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne, which provides detailed explanations of process management, thread scheduling, synchronization mechanisms, and concurrency control. Another important reference is *Modern Operating Systems* by Andrew S. Tanenbaum and Herbert Bos, which discusses operating system design principles and practical implementations of threads and synchronization techniques. In addition, Linux manual pages (man pages) were consulted to understand the usage of POSIX thread libraries, mutexes, semaphores, and condition variables available in Linux environments.

3 Description of Concepts

3.1 Multithreading

Multithreading is the capability of a process to execute multiple threads concurrently. Each thread represents an independent path of execution while sharing the same address space, code, data, and system resources of the parent process. Since threads share resources, communication between them is efficient and requires less overhead than communication between separate processes.

Multithreading is widely used in modern operating systems to improve application performance and responsiveness. On multicore processors, multiple threads can execute in parallel, making better use of available CPU resources.

3.1.1 Advantages of Multithreading

- **Responsiveness:** Allows applications to remain responsive even when some threads are performing lengthy operations.
- **Resource Sharing:** Threads share memory and resources of the same process, enabling efficient communication.
- **Economy:** Creating and managing threads is less expensive than creating processes.
- **Scalability:** Multithreaded applications can effectively utilize multicore processors for improved performance.

Table 1: Thread vs Process

Feature	**Thread**	**Process**
Memory	Shared	Separate
Creation Cost	Low	High
Context Switch	Fast	Slow
Communication	Direct (Shared Memory)	IPC Required

3.2 Race Condition and Critical Section

A race condition occurs when two or more threads access and modify shared data simultaneously, causing the program's output to depend on the order of thread execution. Such situations can lead to inconsistent or incorrect results.

The portion of a program where shared resources are accessed is called the critical section. Only one thread should be allowed to execute its critical section at a time. A correct solution to the critical section problem must satisfy the following requirements:

- **Mutual Exclusion:** Only one thread can access the critical section at a time.
- **Progress:** Threads not in the critical section should not prevent others from entering it.
- **Bounded Waiting:** Every thread should get a fair opportunity to enter the critical section without indefinite delay.

3.3 Mutex

A mutex (Mutual Exclusion Lock) is a synchronization mechanism used to ensure exclusive access to a shared resource. Only one thread can hold a mutex at a given time, while other threads attempting to acquire it must wait until it becomes available.

Mutexes are commonly used to protect critical sections and prevent race conditions. By locking a mutex before accessing shared data and unlocking it afterward, threads can safely coordinate access to common resources.

3.4 Semaphore

A semaphore is a synchronization variable used to manage access to shared resources. It maintains an integer value that indicates the number of available resources.

Semaphores support two atomic operations:

- **Wait (P):** Decreases the semaphore value and blocks the thread if the resource is unavailable.
- **Signal (V):** Increases the semaphore value and wakes a waiting thread if necessary.

Semaphores can be classified as:

- **Binary Semaphore:** Allows only one thread to access a resource at a time.
- **Counting Semaphore:** Allows multiple instances of a resource to be shared among threads.

Semaphores are widely used in synchronization problems such as producer-consumer systems, readers-writers problems, and resource allocation management.

3.5 Condition Variable

A condition variable is a synchronization mechanism that enables threads to wait until a particular condition becomes true. Instead of continuously checking the condition, a thread can enter a waiting state and resume execution only when another thread signals that the condition has been satisfied.

Condition variables are generally used together with mutexes to coordinate thread execution efficiently. They help avoid busy waiting and improve overall system performance by allowing threads to sleep until they are needed.

Table 2: Hardware and Software Requirements

Item	Details
Processor	Intel Core i5 (4 Cores) or Equivalent
RAM	8 GB or Higher
Operating System	Ubuntu 22.04 / WSL2
Compiler	GCC 11.0 or Higher
Library	POSIX Threads (`pthread`), `semaphore.h`
Editor	Visual Studio Code / Vim

4 Methodology and System Design

4.1 Approach

The project is divided into five modules, each demonstrating a different synchronization concept and executed independently for better understanding and analysis.

1. **Module 1: Race Condition Demonstration**
   Demonstrates concurrent access to shared data without synchronization and compares it with mutex-protected execution.
2. **Module 2: Producer–Consumer Problem**
   Uses semaphores and mutexes to coordinate producer and consumer threads operating on a bounded buffer.
3. **Module 3: Condition Variable Signalling**
   Demonstrates ordered execution and thread communication using condition variables.

4. **Module 4: Dining Philosophers Problem**
   Implements resource-sharing among multiple threads using mutexes while avoiding deadlock through resource ordering.

5. **Module 5: Readers–Writers Problem**
   Allows multiple readers to access shared data simultaneously while ensuring exclusive access for writers using semaphores and mutexes.

4.2 Algorithm (Producer–Consumer)

1. Initialize `empty` to BUFFER_SIZE, `full` to 0, and `mutex` to the unlocked state.
2. Producer thread waits for an empty slot in the buffer.
3. Producer acquires the mutex lock and inserts an item into the buffer.
4. Producer releases the mutex and signals that a new item is available.
5. Consumer thread waits until an item becomes available.
6. Consumer acquires the mutex lock and removes an item from the buffer.
7. Consumer releases the mutex and signals that a buffer slot has become empty.
8. Steps continue until all items are produced and consumed.

4.3 Multithreading

Advantages: responsiveness, resource sharing, economy, scalability on multicore CPUs. Compare briefly with multiprocessing (Table 3).

Table 3: Thread vs Process

Feature	**Thread**	**Process**
Memory	Shared	Separate
Creation cost	Low	High
Context switch	Fast	Slow
Communication	Direct (shared memory)	IPC needed

4.4 Race Condition and Critical Section

A race condition occurs when the result depends on the order in which threads execute. The code segment that accesses shared data is the critical section. A correct solution needs: mutual exclusion, progress and bounded waiting.

4.5 Mutex

A binary lock: only one thread can hold it. APIs: `pthread_mutex_lock()`, `pthread_mutex_unlock()`.

4.6 Semaphore

An integer variable accessed by `wait()` (P) and `signal()` (V). Counting semaphores manage multiple identical resources. APIs: `sem_init()`, `sem_wait()`, `sem_post()`.

4.7 Condition Variable

Lets a thread sleep until a condition becomes true. APIs: `pthread_cond_wait()`, `pthread_cond_signal()`, `pthread_cond_broadcast()`.

5 Implementation

5.1 Module 1: Race Condition and Mutex

File: `module1_race_condition.c`

```c
/* module1_race_condition.c
 * Demonstrates a race condition and its fix using a mutex.
 * Compile: gcc module1_race_condition.c -o module1 -lpthread
 * Run:     ./module1
 */
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 8
#define INCREMENTS  1000000

volatile long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *unsafe_inc(void *arg) {
    for (int i = 0; i < INCREMENTS; i++)
        counter++;                      /* race condition */
    return NULL;
}

void *safe_inc(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&lock);
        counter++;                      /* critical section */
        pthread_mutex_unlock(&lock);
    }
    return NULL;
```

```
}

void run(void *(*fn)(void *), const char *label) {
    pthread_t t[NUM_THREADS];
    counter = 0;
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&t[i], NULL, fn, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(t[i], NULL);
    printf("%s: expected = %d, actual = %ld\n",
           label, NUM_THREADS * INCREMENTS, counter);
}

int main(void) {
    printf("=== Module 1: Race Condition vs Mutex ===\n");
    run(unsafe_inc, "Without mutex");
    run(safe_inc,   "With mutex   ");
    return 0;
}
```

5.2 Module 2: Producer–Consumer using Semaphores

File: module2_producer_consumer.c

```
/* module2_producer_consumer.c
 * Bounded-buffer Producer-Consumer using semaphores and a mutex.
 * Compile: gcc module2_producer_consumer.c -o module2 -lpthread
 * Run:     ./module2
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Produced: %d at slot %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
```

```
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from slot %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        usleep(250000);
    }
    return NULL;
}

int main(void) {
    pthread_t p, c;
    printf("=== Module 2: Producer-Consumer (Semaphores) ===\n");
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    printf("Done.\n");
    return 0;
}
```

5.3 Module 3: Condition Variables

File: `module3_condition_variable.c`

```
/* module3_condition_variable.c
 * A thread waits on a condition variable until another thread signals
    it.
 * Compile: gcc module3_condition_variable.c -o module3 -lpthread
 * Run:     ./module3
 */
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cv = PTHREAD_COND_INITIALIZER;
```

```c
int ready = 0;

void *waiter(void *arg) {
    pthread_mutex_lock(&m);
    printf("Waiter: waiting for condition...\n");
    while (!ready)                          /* guards against spurious
        wakeups */
        pthread_cond_wait(&cv, &m);
    printf("Waiter: condition met, proceeding\n");
    pthread_mutex_unlock(&m);
    return NULL;
}

void *signaller(void *arg) {
    sleep(2);                               /* simulate work */
    pthread_mutex_lock(&m);
    ready = 1;
    printf("Signaller: setting condition and signalling\n");
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&m);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    printf("=== Module 3: Condition Variable ===\n");
    pthread_create(&a, NULL, waiter, NULL);
    pthread_create(&b, NULL, signaller, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

5.4 Module 4: Dining Philosophers (Deadlock Avoidance)

Five philosophers sit around a table with one fork between each pair. A philosopher needs both adjacent forks to eat. If everyone picks up the left fork first, a circular wait (deadlock) can occur. Here every philosopher picks the lower-numbered fork first, which breaks the circular-wait condition.

File: `module4_dining_philosophers.c`

```c
/* module4_dining_philosophers.c
 * Dining Philosophers with deadlock avoidance by resource ordering.
 * Compile: gcc module4_dining_philosophers.c -o module4 -lpthread
 * Run:     ./module4
 */
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```c

#define N 5
#define ROUNDS 3

pthread_mutex_t fork_m[N];

void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left = id, right = (id + 1) % N;
    int first  = left < right ? left : right;   /* lower-numbered fork first */
    int second = left < right ? right : left;

    for (int r = 1; r <= ROUNDS; r++) {
        printf("Philosopher %d is thinking\n", id);
        usleep(100000);

        pthread_mutex_lock(&fork_m[first]);
        pthread_mutex_lock(&fork_m[second]);

        printf("Philosopher %d is eating (round %d)\n", id, r);
        usleep(150000);

        pthread_mutex_unlock(&fork_m[second]);
        pthread_mutex_unlock(&fork_m[first]);
    }
    return NULL;
}

int main(void) {
    pthread_t t[N];
    int ids[N];
    printf("=== Module 4: Dining Philosophers ===\n");
    for (int i = 0; i < N; i++)
        pthread_mutex_init(&fork_m[i], NULL);
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&t[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);
    for (int i = 0; i < N; i++)
        pthread_mutex_destroy(&fork_m[i]);
    printf("All philosophers finished without deadlock.\n");
    return 0;
}
```

5.5 Module 5: Readers–Writers Problem

Many readers may read shared data at the same time, but a writer needs exclusive access. A mutex protects the reader count, and a semaphore `rw_mutex` gives writers (and the first reader) exclusive access. This is the reader-preference version, so writers may starve if readers keep arriving.

File: `module5_readers_writers.c`

```c
/* module5_readers_writers.c
 * Readers-Writers (reader preference) using a semaphore and a mutex.
 * Compile: gcc module5_readers_writers.c -o module5 -lpthread
 * Run:     ./module5
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int shared_data = 0, read_count = 0;
pthread_mutex_t rc_mutex = PTHREAD_MUTEX_INITIALIZER;
sem_t rw_mutex;

void *reader(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&rc_mutex);
        read_count++;
        if (read_count == 1) sem_wait(&rw_mutex);   /* first reader
           blocks writers */
        pthread_mutex_unlock(&rc_mutex);

        printf("Reader %d reads value = %d\n", id, shared_data);
        usleep(100000);

        pthread_mutex_lock(&rc_mutex);
        read_count--;
        if (read_count == 0) sem_post(&rw_mutex);   /* last reader
           releases */
        pthread_mutex_unlock(&rc_mutex);
        usleep(100000);
    }
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 3; i++) {
        sem_wait(&rw_mutex);
        shared_data++;
        printf("Writer %d wrote value = %d\n", id, shared_data);
```

```
        usleep(150000);
        sem_post(&rw_mutex);
        usleep(200000);
    }
    return NULL;
}

int main(void) {
    pthread_t r[3], w[2];
    int rid[3] = {1, 2, 3}, wid[2] = {1, 2};
    printf("=== Module 5: Readers-Writers ===\n");
    sem_init(&rw_mutex, 0, 1);
    for (int i = 0; i < 3; i++) pthread_create(&r[i], NULL, reader, &
        rid[i]);
    for (int i = 0; i < 2; i++) pthread_create(&w[i], NULL, writer, &
        wid[i]);
    for (int i = 0; i < 3; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    sem_destroy(&rw_mutex);
    printf("Done.\n");
    return 0;
}
```

6 Results and Output

All programs were compiled with `gcc <file>.c -o <name> -lpthread` and run on Ubuntu Linux.

6.1 Module 1 Output

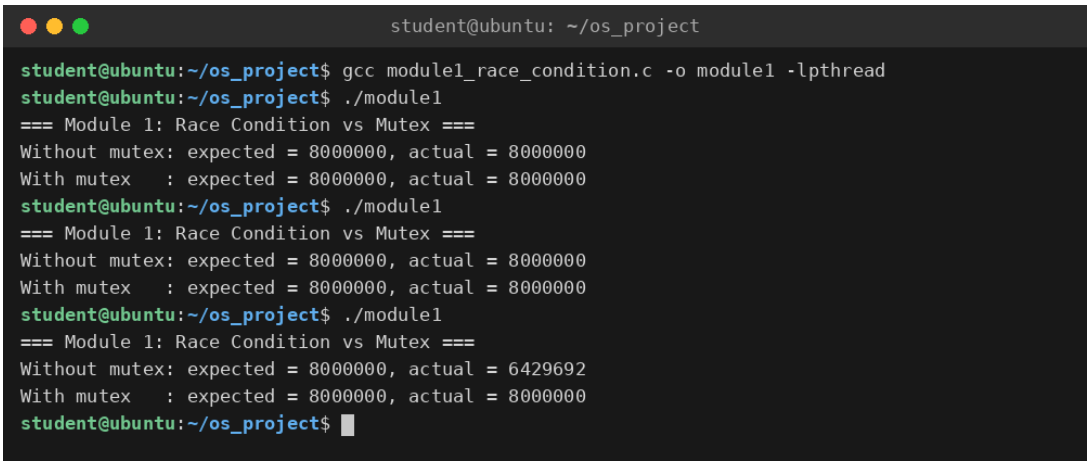


Figure 1: Race Condition

6.2 Module 2 Output

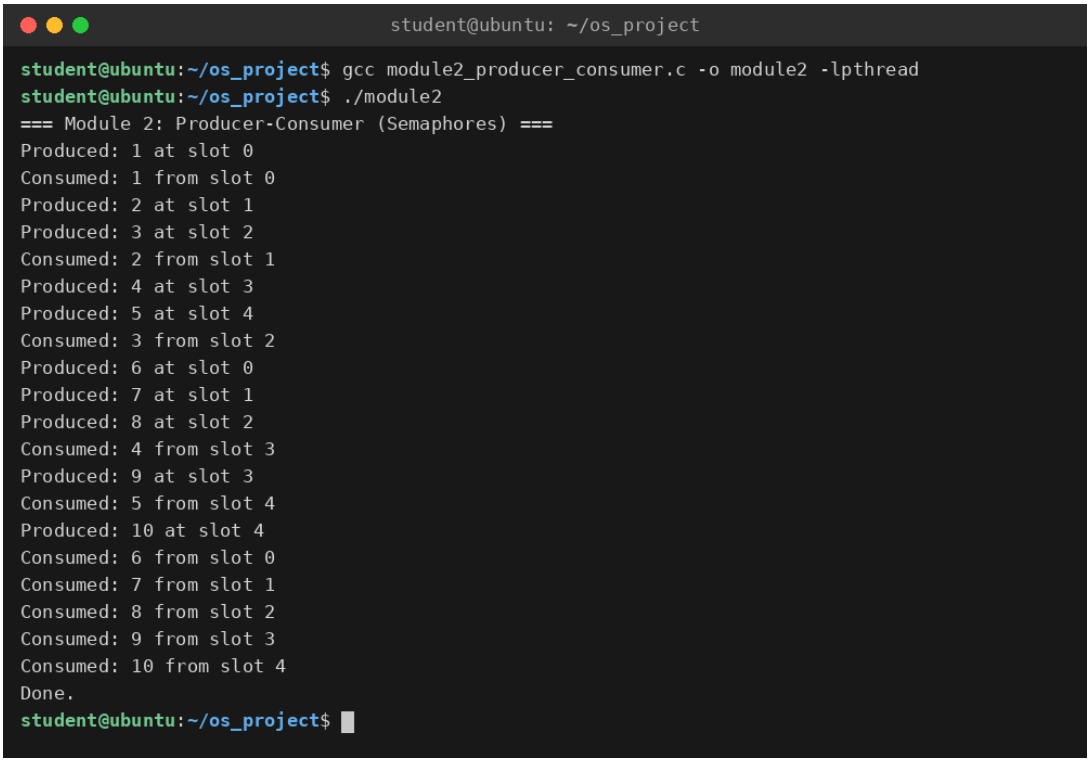


Figure 2: producer and consumer

6.3 Module 3 Output

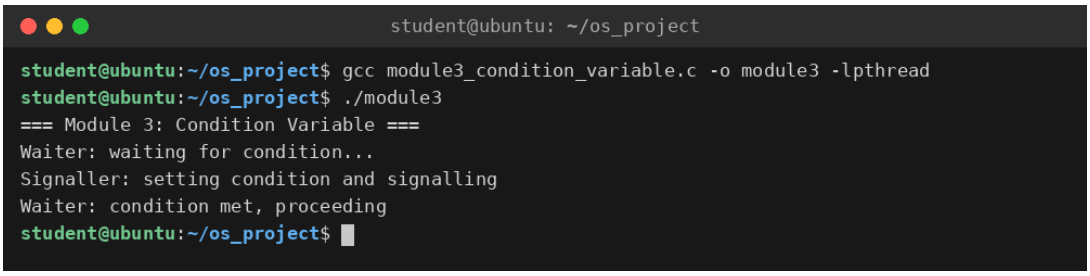


Figure 3: condition variable demo

6.4 Module 4 Output

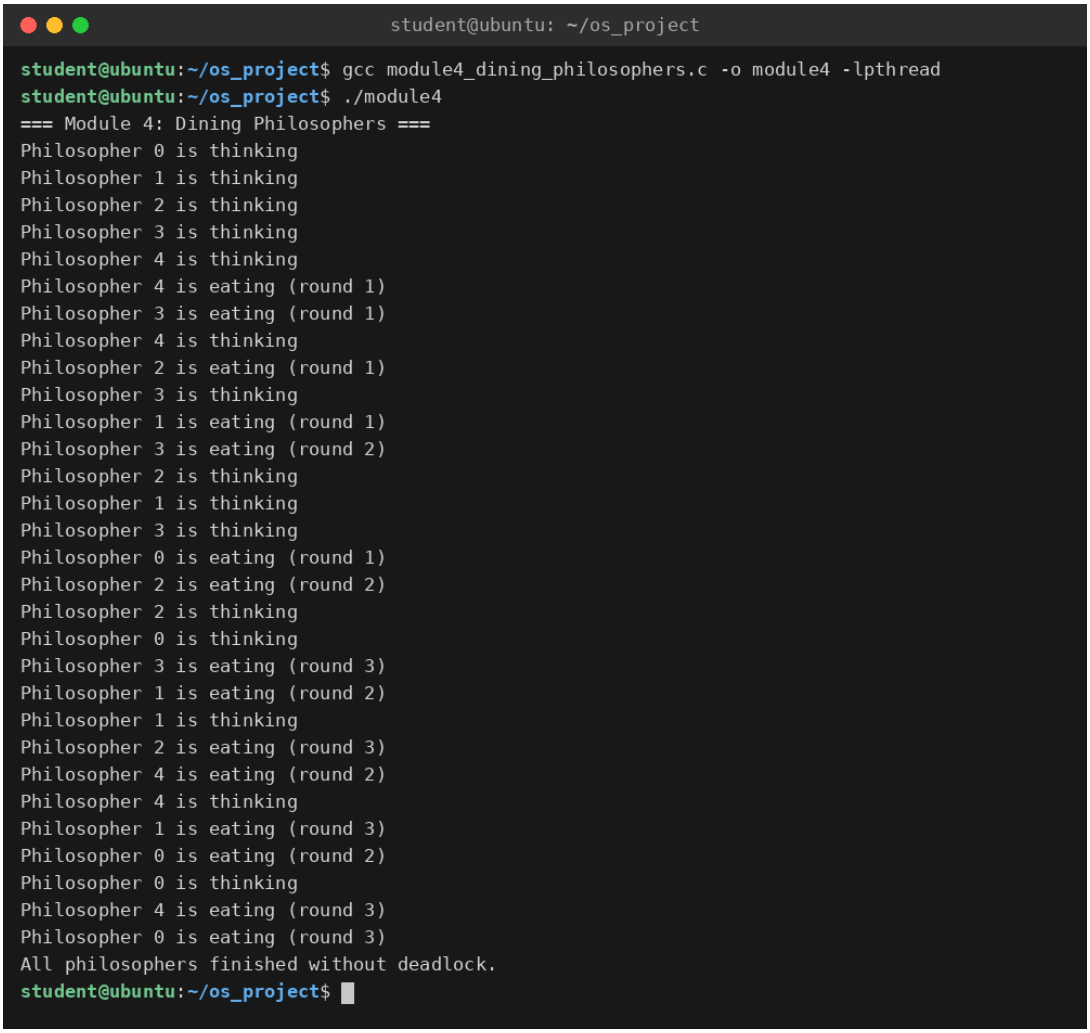


Figure 4: Dining Philosophers

6.5 Module 5 Output

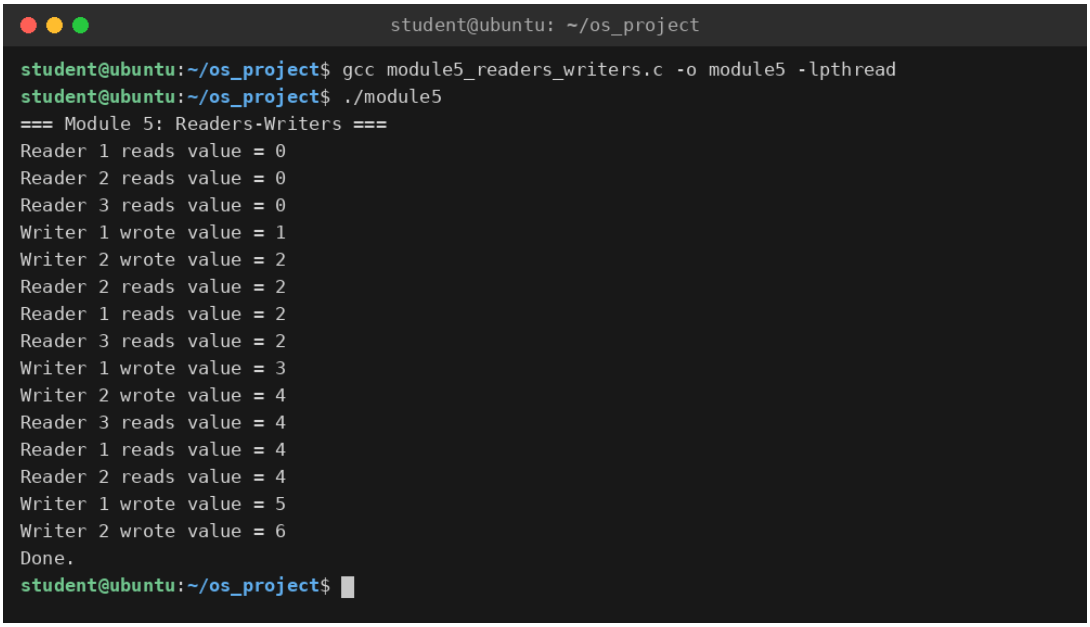


Figure 5: Readers-Writers

6.6 Comparison Table

Module 1 runs the same counting task twice: once without any lock and once protected by a mutex. Eight threads each increment a shared counter 1,000,000 times, so the correct final value is $8 \times 1{,}000{,}000 = 8{,}000{,}000$. Table 4 records the three runs shown in Figure 1.

Table 4: Observed results of Module 1 (8 threads × 1,000,000 increments)

Run	Expected	Without Mutex	With Mutex
1	8000000	8000000	8000000
2	8000000	8000000	8000000
3	8000000	6429692	8000000

Explanation of the table.

- **With mutex:** the counter equals the expected value in every run. The mutex lets only one thread execute `counter++` at a time, so no update is lost.
- **Without mutex:** `counter++` is really three steps (read, add, write back). When two threads read the same old value, one increment is overwritten. In run 3 the final value is 6429692, so $8000000 - 6429692 = 1570308$ increments were lost.
- **Why runs 1 and 2 are correct:** a race condition is not guaranteed to show up on every run. It depends on how the scheduler interleaves the threads, so the

unsynchronised version can give the right answer by chance. This is why race conditions are dangerous: the program looks correct in some runs and fails in others.

Table 5 summarises all five programs of this project: the file, the synchronisation mechanism, the shared resource it protects and the result obtained.

Table 5: Summary of project programs and their results

No.	File	Mechanism	Shared resource	Result observed
1	`module1_race_condition.c`	Mutex	Counter	Without mutex the count was wrong in run 3 (6429692); with mutex it was always 8000000.
2	`module2_producer_consumer.c`	Two semaphores and a mutex	Bounded buffer (5 slots)	All 10 items produced and consumed in order; no overflow or underflow.
3	`module3_condition_variable.c`	Condition variable and mutex	Flag `ready`	Waiter slept until the signaller set the flag, then proceeded.
4	`module4_dining_philosophers.c`	Five mutexes with resource ordering	Five forks	All 5 philosophers ate 3 times each; no deadlock.
5	`module5_readers_writers.c`	Semaphore and mutex	`shared_data`, `read_count`	Readers read together; each writer had exclusive access; final value 6 (2 writers $\times$ 3 writes).

7 Mapping POs and SDGs

7.1 Program Outcomes (POs) Mapping

Table 6: Mapping of Program Outcomes to Project Relevance

PO	Program Outcome	Project Relevance
PO1	Engineering Knowledge	Applies operating system concepts related to processes, threads, scheduling, and synchronization.
PO2	Problem Analysis	Analyzes issues such as race conditions, deadlocks, and resource sharing in concurrent systems.
PO3	Design/Development of Solutions	Develops multithreaded applications using synchronization mechanisms to ensure correct execution.
PO4	Conduct Investigations	Studies thread behavior and compares system performance with and without synchronization.
PO5	Engineering Tool Usage	Uses Linux, POSIX Threads (Pthreads), GCC compiler, and system monitoring tools for implementation and testing.

7.2 Sustainable Development Goals (SDGs) Mapping

Table 7: Mapping of SDGs to Project Relevance

SDG	Goal	Project Relevance
SDG 4	Quality Education	Enhances practical understanding of operating system concepts through hands-on implementation of multithreading and synchronization.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates efficient resource utilization and concurrent processing techniques used in modern computing systems.

8 Conclusion

The Multithreading and Synchronization project successfully demonstrates the concepts of concurrent execution and resource sharing in operating systems. The implementation shows how multiple threads can execute simultaneously to improve system responsiveness and performance. The project also highlights the importance of synchronization techniques such as mutex locks and semaphores in preventing race conditions and ensuring data consistency. Through practical experimentation, the project provides a clear understanding of thread creation, management, synchronization, and inter-thread communication. Overall, the project serves as an effective learning platform for understanding the role of multithreading in modern operating systems and real-world applications.

9 Future Work

- Implementation of advanced synchronization techniques such as condition variables and read-write locks.
- Deadlock detection and prevention mechanisms.
- Performance analysis using different scheduling algorithms.
- Development of producer-consumer and reader-writer problem simulations.
- Integration of thread pools for efficient task management.
- Visualization of thread execution and synchronization events through a graphical interface.
- Support for distributed and parallel computing environments.

10 Bibliography

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, Wiley.

2. Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson.

3. W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, Addison-Wesley.

4. POSIX Threads Documentation, https://man7.org/linux/man-pthreads.7.html.

5. Linux Programmer's Manual, https://www.kernel.org/pub/linux/docs/man-pages/.